

Тема 5. Настройване и тестване на ПО

дисциплина: Технология на софтуерното производство
лектор: доц. В.Божикова

Съдържание на лекцията:

1. Откриване на грешки в програмите

1.1. Основни понятия

1.2. Видове грешки

1.3. Източници на грешките

1.4. Основни дейности за откриване и отстраняване на грешки

2. Подходи и насоки за настройка на програмите

2.1. Подходи за ръчно локализиране на грешките

2.2. Автоматизация на настройката

2.3. Общи насоки за ръчна настройка на програмите

3. Тестване на софтуер

3.1. Каква е целта на софтуерното тестване?

3.2. Аксиоми на тестването (фундаменталните принципи за тестването)

4. Методи за тестване.

4.1. Класификация на методите за тестване в зависимост от нивото на тестване (т.е каква част от програмната система се тества):

4.2. Класификация на методите за тестване в зависимост от това, кой извършва тестването:

4.3. Класификация на методите за тестване в зависимост от това дали се взема предвид или се игнорира вътрешната структура на софтуера:

4.4. Класификация на методите за тестване в зависимост от избраните тестови данни и очаквани резултати:

4.5. Класификация на методите за тестване в зависимост от подаваните стойности на входните данни:

4.6. Класификация на методите за тестване в зависимост от начина за интеграция на модулите в системата

4.7. Класификация на методите за тестване в зависимост от целта на тестването

4.8. Кой метод е по-добър

5. Автоматизиране на тестването

7. Осъществяване на тестването

Допълнителните дейности в софтуерния процес

- Процесът на създаване на софтуер може да се разглежда като съвкупност от:
 - **основни дейности** (дефиниране на изискванията, проектиране, програмиране) и
 - **допълнителни дейности**:
 - **настройване и тестване;**
 - **управление на проекта;**
 - **осигуряване на качеството;**
 - **измерване;**
 - **документиране;**
 - **управление на софтуерните конфигурации.**
- Характерно за допълнителните дейности е, че те са „фонови“ и се осъществяват в продължение на няколко или дори всички фази на жизнения цикъл.

- **Настройване и тестване се явяват допълнителни дейности, те са „фонови“ - осъществяват се в продължение на няколко фази на жизнения цикъл.**
- **Това са основните дейности, свързани с откриването и отстраняване на грешки в програмите.**

1. Откриване на грешки в програмите

Откриването на грешки е част от цялостното осигуряване на качеството на софтуера.

1.1. Основни понятия

- **Качеството на програмните продукти е абстрактно понятие.** Твърди се, че то трудно се дефинира, но е лесно да се разпознае при реалното използване на съответния програмен продукт. Една от трудностите при дефинирането е **относителният характер** на качеството. Могат да се избират различни дефиниции, в зависимост от интересувания ни аспект на качеството, от това кой го оценява и кои са обектите, чието качество се изследва.
- Ще се придържаме към следната дефиниция на качество на софтуер ([1],[2]) :

- **Качествен** е този програмен продукт, който удовлетворява формулираните към него изисквания.

Какво е **дефект** в програмата?

- Всяко отклонение от изискванията ще наричаме **дефект**. Дефектите обикновено се дължат на една или няколко **грешки**.

Какво е **грешка** в програмата?

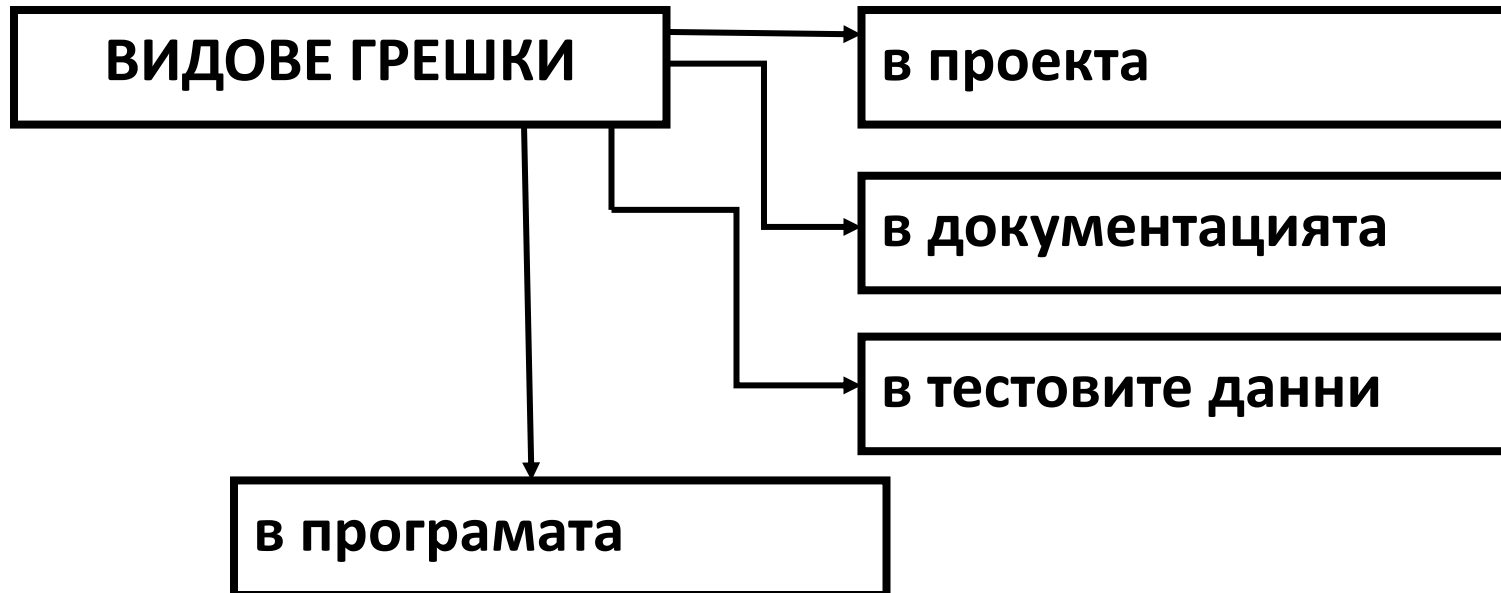
- Дефиниция в [1],[2]: Под грешка ще разбираме **неправилност, отклонение или неволно преиначаване на обект или процес**.
- Определение на Майерс: „В програмата има грешки, ако тя не изпълнява това, което потребителят разумно очаква от нея”, тоест ако тя не изпълнява потребителските изисквания.

1.2. Видове грешки

- **Налице са различни класификации на грешките в програмите.**
- **Защо е важно да се класифицират грешките? Защото, в зависимост от вида на грешките се прилагат различни техники за търсенето им.**

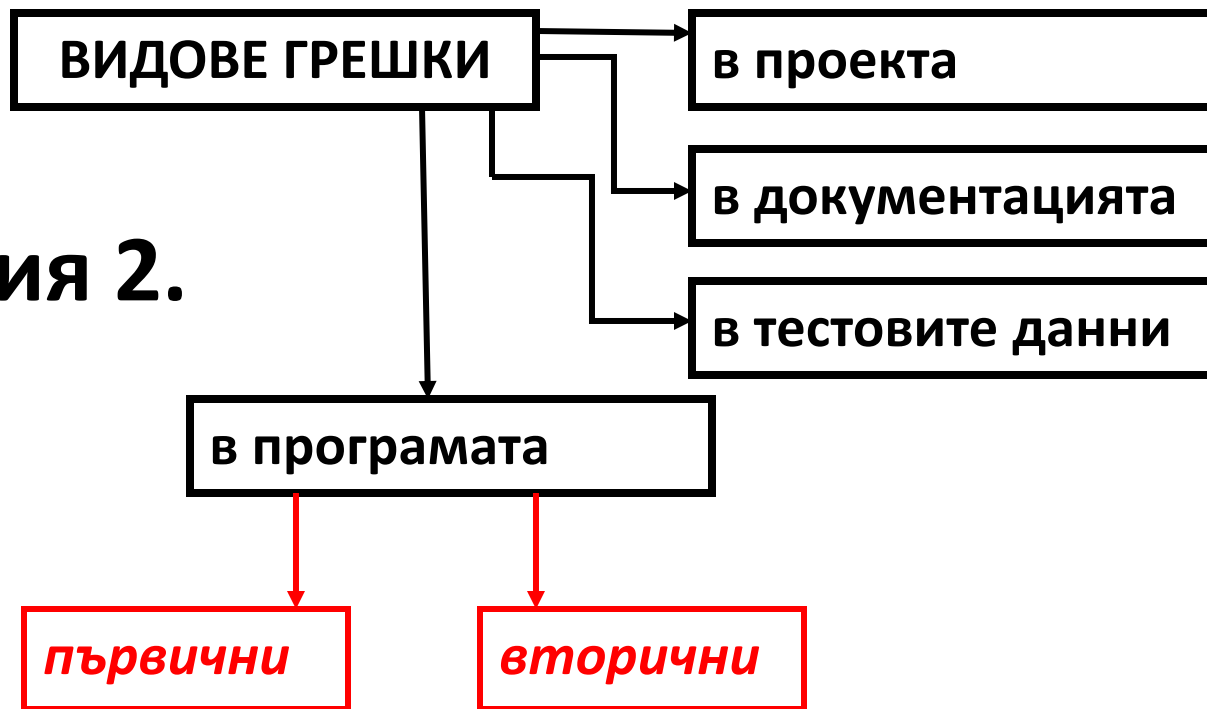
Класификация 1.

- В зависимост от това, кога в ЖЦ на софтуерния продукт се откриват, грешките могат да бъдат:.



- По-нататък ще се фокусираме само върху грешките в програмите, също - върху дейностите за откриването и отстраняването им, както и върху средствата, подпомагащи тези дейности.

Класификация 2.



Грешките могат да бъдат:

- **първични** — неправилности в текста на програмите, подлежащи на непосредствено коригиране;
- **вторични** — изкривявания на получените резултати (в резултат на зацикляне например), дължащи се на първична грешка).

Класификация 3.

ВИДОВЕ ГРЕШКИ

в проекта

в документацията

в тестовите данни

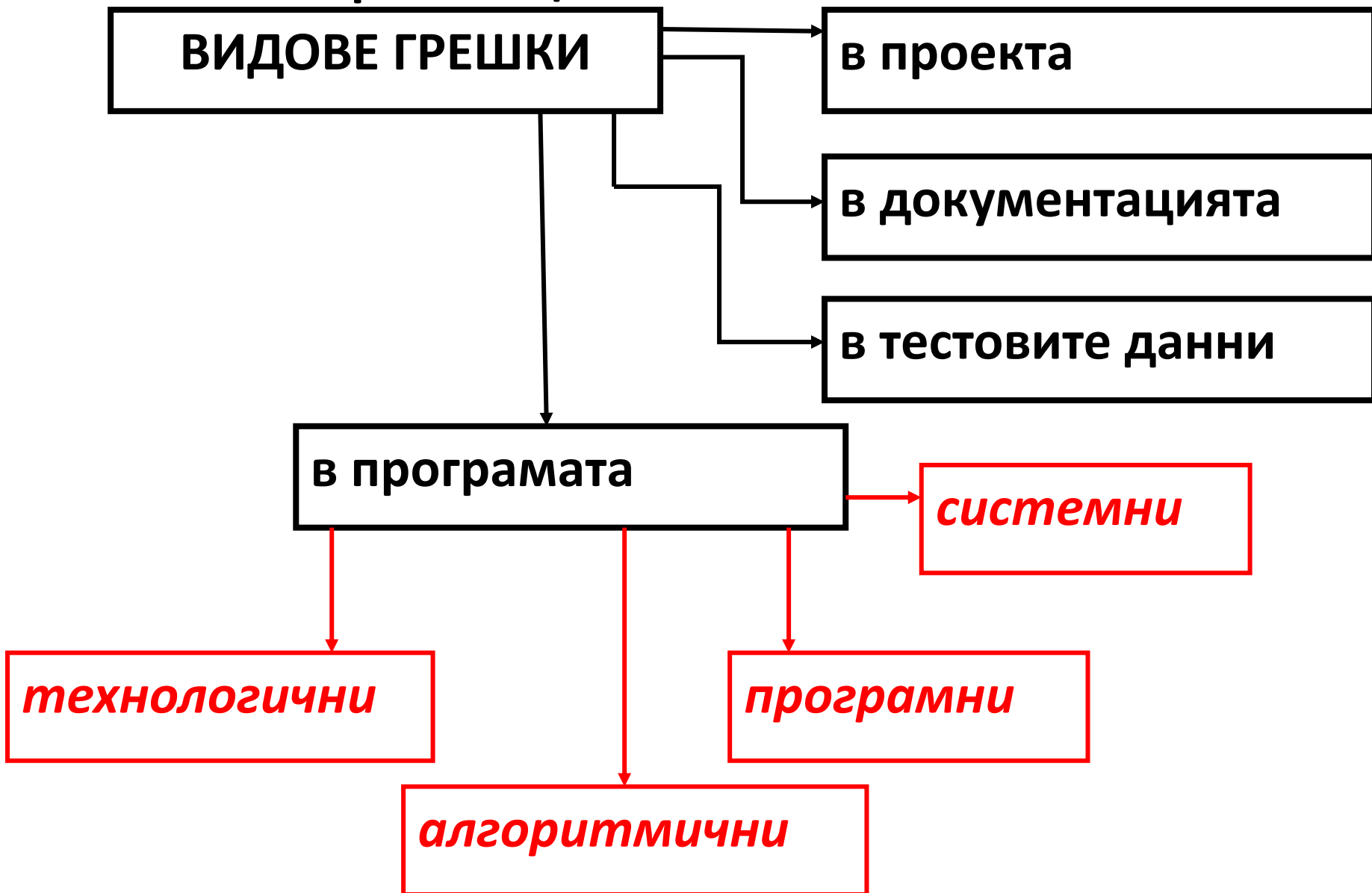
в програмата

системни

технологични

програмни

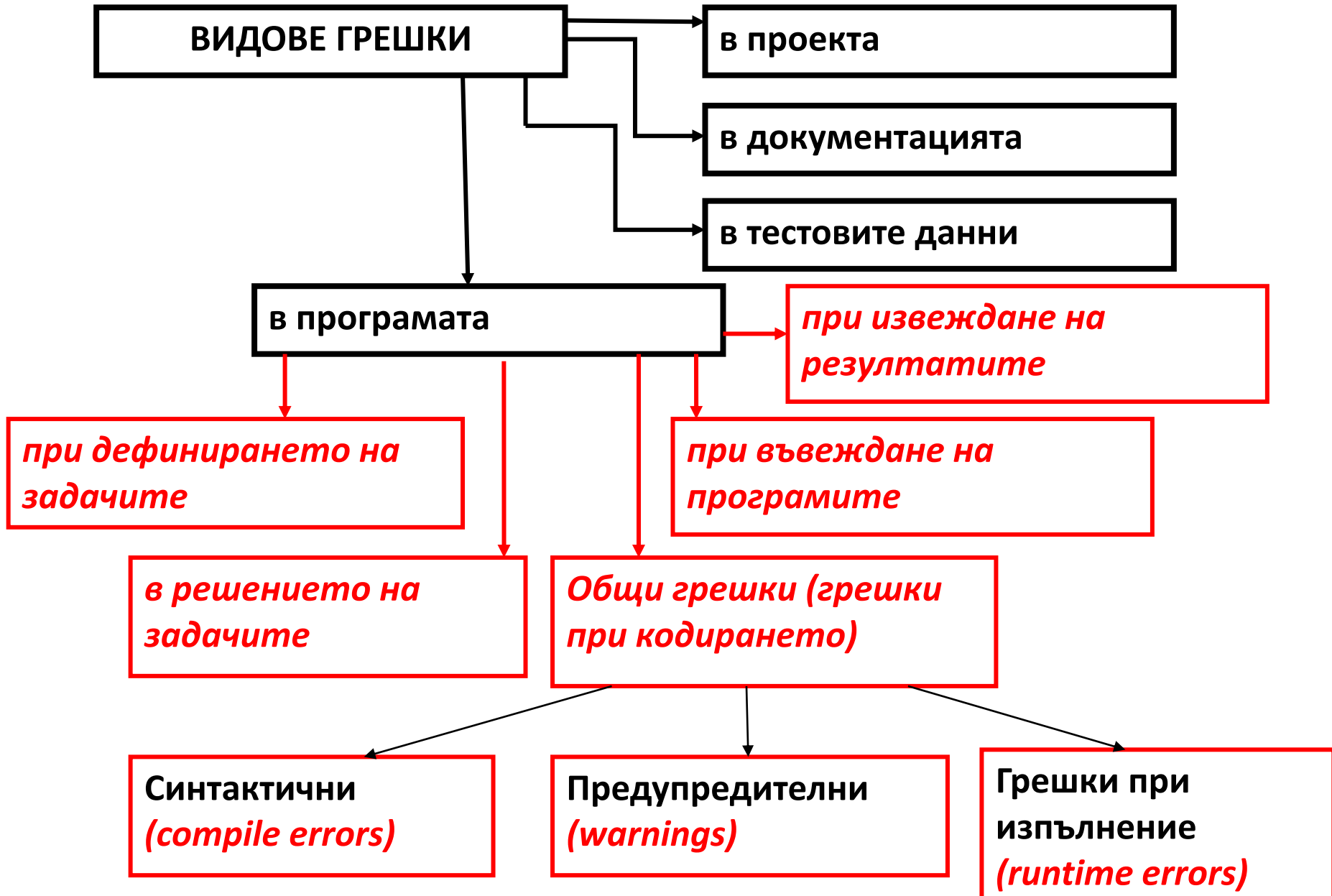
алгоритмични



Съгласно тази класификация 3 грешките могат да бъдат:

- а) **технологични** грешки — при въвеждане на програмите или при подготовка на входните данни върху технически носители;
- б) **алгоритмични** грешки;
- в) **програмни** грешки — неправилно използване на конструкциите от съответните езици за програмиране;
- г) **системни** — свързани с функциониране в определена операционна система.

Класификация 4.



Съгласно класификация 4 имаме:

- **Грешки при дефинирането на задачите**

- Лошото поставяне на задачата води до програма, която решава правилно една погрешна задача. Изходът от тази ситуация е повторно писане на програмата и ново заделяне на ресурси.

- **Грешки в решението на задачата (логически грешки)**

- Задачата е дефинирана правилно, но има грешки, произлизащи от неправилен (или лошо избран) алгоритъм за решение на задачата.

- **Общи грешки (грешки при програмирането)**
 - Задачата е дефинирана правилно, избран е подходящ алгоритъм за решаване, но са налице грешки, свързани с кодирането на алгоритъма:
 - **Синтактични грешки** (разпознаваеми са като такива от компилатора, **compile errors**): липсващ необходим знак за пунктуация; несъответствие в броя на скобите; липсващи скоби и др. Грешка, причинена от неправилни операции върху данни, например изпълнение на математическа операция върху символни данни.
 - **Предупредителни грешки (warnings)**: компилаторът може да предупреди за нещо необичайно (вероятно невярно) например, използване на неинициализирани данни или неизползвани данни. Програмистът сам взема решение дали да игнорира или не този тип грешки.

- **Грешки при изпълнение (runtime errors)** : Синтактично правилни оператори могат да причинят **грешки при изпълнение (бъгове)**:
 - **аритметични грешки** (деление на 0, загуба на точност и др.);
 - **логически грешки** (безкраен цикъл или рекурсия и др.);
 - **ресурсни грешки** (липсващи данни, например файл; неправилно подредени данни; неправилен формат на данните и др.; достъпване на забранена памет - segmentation fault; недостиг на място в определено пространство от паметта - buffer overflow и др.).

Според времето си на проявление, грешките при изпълнение могат да са:

- Постоянни, при всяко пускане;
- Понякога («плаващ» тип);
- Само на машината на клиента (зависят от локалните настройки у клиента);

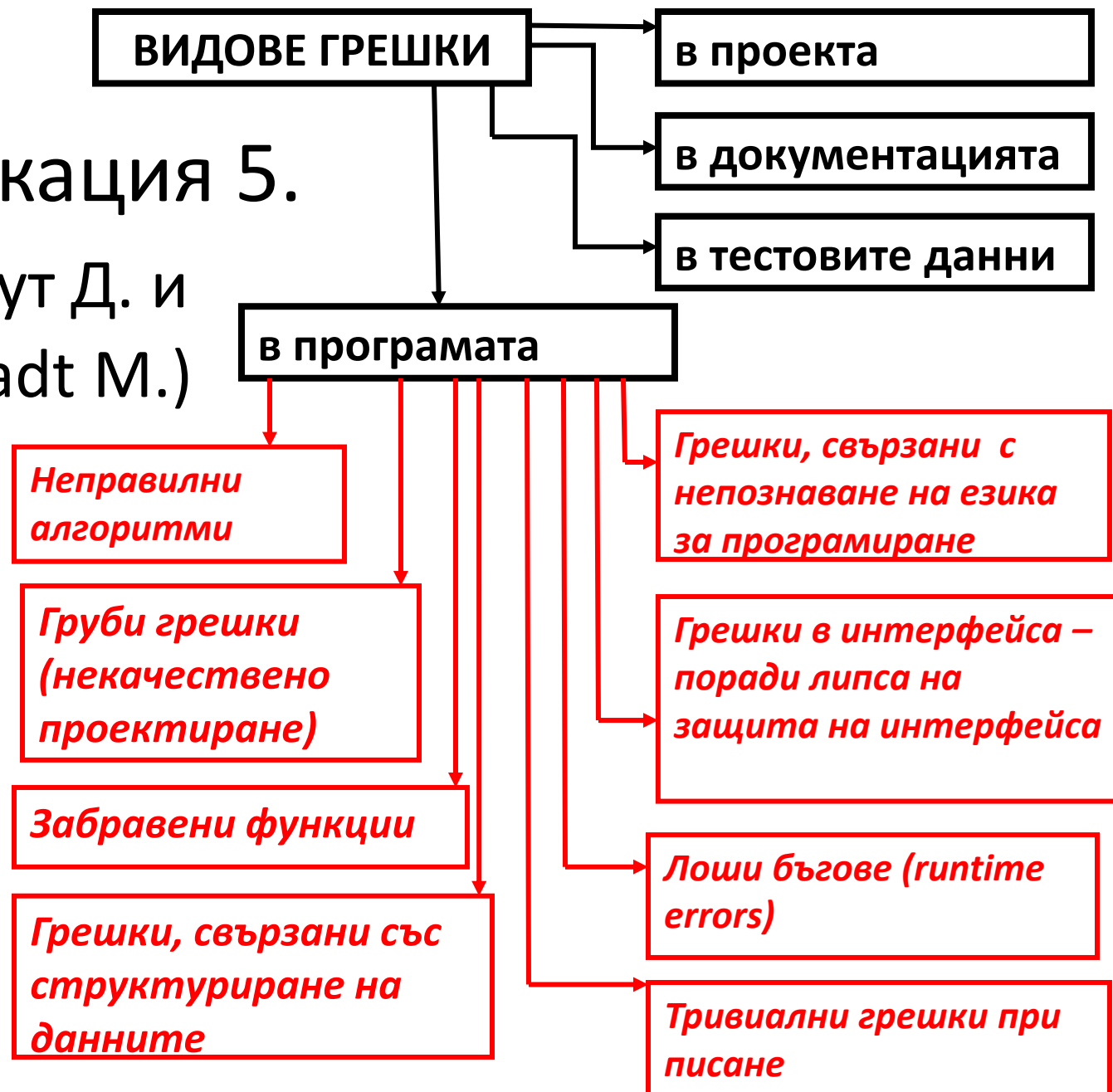
- *Грешки при въвеждане на програмите*
 - грешки главно поради невнимателно и двусмислено изписване на някои символи, например буква О вместо 0 (нула).
- *Грешки при извеждане на резултатите*

Тук грешките могат да възникнат главно по две причини:

- неправилен формат на извежданите резултати;
- недостатъчен брой позиции вдясно от десетичната точка.

Класификация 5.

(на Кнут Д. и
Eisenstadt М.)



1.3. Причини за грешките

- **Причините за грешки** (съгласно анализа на статистическите данни), попадат в една от следните **9 категории**:

	Категория на източника на грешките	Относителен дял, %
1	Mem (Съсипване или изчерпване на паметта)	26
2	Vendor (Проблеми в използвания софтуер и хардуер, например грешки в компилатора)	18
3	Dec. logic (Пропуски в проектирането на логиката)	13
4	Init (Грешна инициализация, неправилни типове, дефиниции и декларации)	12
5	Lex (Лексически проблеми, неправилен разбор, двусмислен синтаксис)	11
6	Var (Неправилни променливи и оператори)	6
7	Unsolved (Самите грешки са открити и отстранени, но причините остават все още неизвестни)	5
8	Lang (Двусмислена семантика на езика или неговото неразбиране)	5
9	Behav (Коварно поведение на крайния потребител (програмист), например натискане на няколко бутона едновременно)	4

1.4. Основни дейности за откриване и отстраняване на грешки

- Двете основни дейности, свързани с откриването и отстраняването на грешки в програмите са **тестване и настройване**.
- **Тестване** (testing) - **изследване** или **изпълнение** на програмите, с цел **установяване на наличие на грешка**.
- **Настройване** (debugging) - **локализиране и отстраняване** на установени грешки.

Настройване/Тестване

- В много случаи не се прави разлика между настройване и тестване. **Тестването определя, че съществува грешка, докато настройката локализира и поправя грешката.**
- **!!!Задачата на етапа настройка е да се създаде работещ вариант на програмата.** Очевидно е, че този работещ вариант не е изчистен в достатъчна степен от грешки, но във всеки случай не съдържа поне такива, които да не позволяват на програмата да бъде стартирана.

Настройване/Тестване

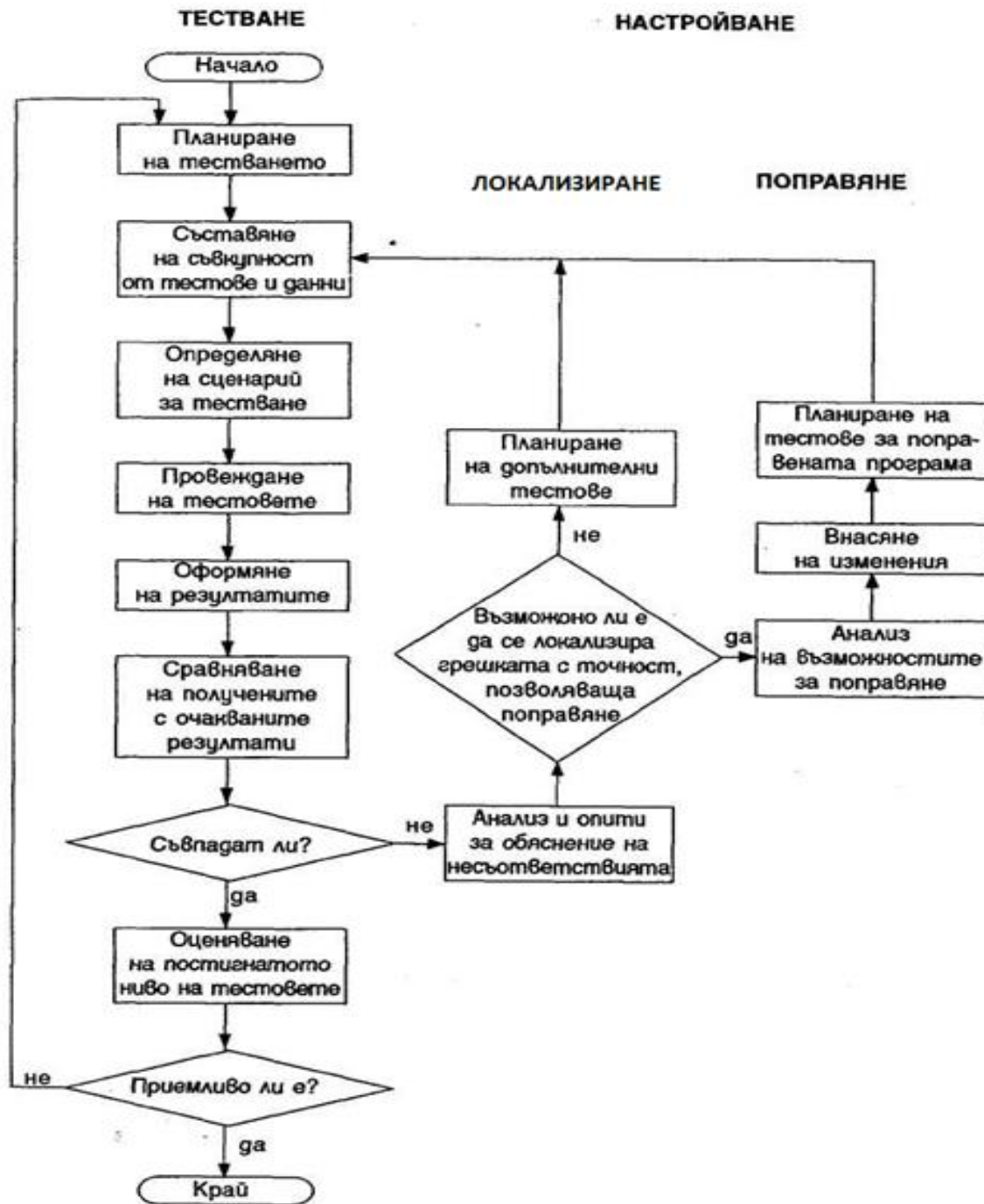
- **Може да се каже, че настройката и тестването са противоположни дейности.**
- **Докато настройката е креативен процес по отношение на програмата, то тестването е разрушителен процес.** При него програмата трябва да се атакува с разнообразни данни, в това число и невероятни, да работи в такива условия, че да се открият (покажат) колкото може повече грешки.

Настройване/Тестване

В резюме:

- Когато настройката е завършена, програмата определено **решава някаква задача**.
- Когато тестването е завършено, програмата **решава изискваната задача**, тоест задачата която се изисква да бъде решена.
- Връзката между тестването, настройването и поправянето на грешки може да се илюстрира чрез схемата:

Тестване/ Настройване



За използваната терминология

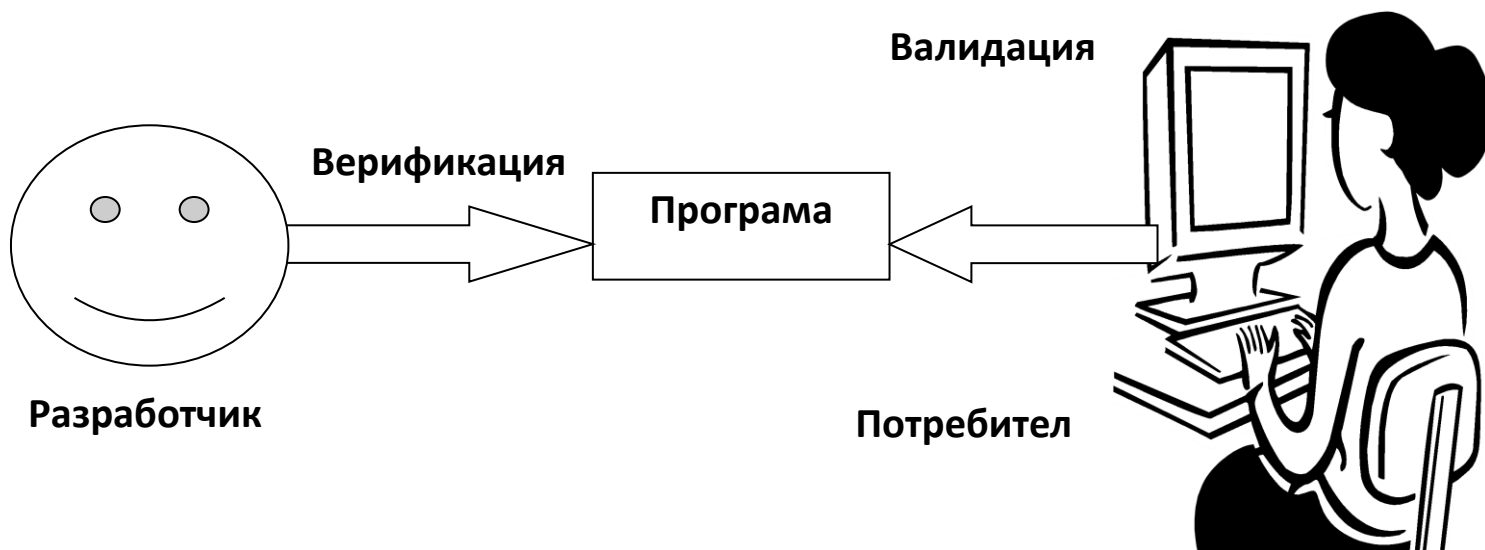
- Следва да се посочи, че в разглежданата област **няма достатъчна яснота по използваната терминология**. Много често термините **тестване**, **настройка**, **верификация**, **валидация**, **доказателство**, **контрол**, **изпитване** и други се използват като синоними и за различните хора имат различен смисъл.
- Може да се направи известна аналогия между понятията **настройка** и **тестване** и **верификация** и **валидация** (V&V, тоест **Verification and Validation**), изхождайки от определения дадени от Боем за V&V.

V&V

- **Валидацията** отговаря на въпроса. Are we building the right program? (Създаваме ли правилния продукт?) Валидацията трябва да покаже, че програмата прави това, което потребителят иска.
- **Верификацията** отговаря на въпроса. Are we building the product right? (Създаваме ли продукта правилно?). Верификацията трябва да покаже, че програмата работи коректно.

V&V (Verification and Validation)

- V&V съдържат в себе си погледа на разработчика (Verification) и погледа на потребителя (Validation) към разработеното приложение:



2. Подходи и насоки за локализиране на грешките

- Задачата на настройката на програмите е да се отстрани онзи минимум от грешки, който не позволява тя да се стартира и изпълни нормално.
- **Настройката може да стане**
 - ръчно и чрез**
 - инструментални средства (дебъгери).**
- **Ръчното локализиране на грешки** в определен смисъл изкуство. Придобиването на навици и умения за извършване на тази дейност е въпрос на време и практически опит.

2.1. Подходи за ръчно локализиране на грешките

- **Събиране и натрупване на сведения.** Получаване на повече информация за работата на продукта, например чрез **вграждане в кода на програмата на оператори за печат** или на **точки на прекъсване (breakpoints)**.
- **Inspection.** Тази дума е хибрид от **inspection** (преглеждане, проучване на програмата), **simulation** (ръчно симулиране) и **speculation** (размишляване върху програмата, анализиране) - 3 подхода, често пъти използвани от програмистите. Тук те се съчетават за търсене на грешки в ПО.

- **Разпознаване (идентифициране) на клишета от експерт.** Тук попадат случаите, в които програмистът извиква цяла група хора на помощ, с идеята че все някой от групата е в състояние да посочи мястото на грешката, прилагайки своя опит и съотнасяйки конкретния случай към някои от тези, с които вече се е сблъсквал. Такова разпознаване на грешките може да се отнесе към **«евристичното локализиране»**.
- **Насочен експеримент.** Когато стане ясна идеята за източника на грешката, може да се извърши **“насочено” изпълнение** на програмата така, че тя да “премине” през подозрителното място.

И др.

Подходи за ръчно локализиране на грешките

	Категория	Относителен дял, %
1	Събиране на данни	55
2	Inspection	26
3	Разпознаване на клишета от експерти	9
4	Насочен експеримент	8
5	Други	2

2.2 Автоматизирани средства за настройка

- Инструменталните средства , подпомагащи настройката и влизащи в състава на съвременните интегрирани среди за програмиране се наричат **дебъгери**.
- Работата с тези средства е специфична за всяка интегрирана среда за програмиране, затова няма да се спираме подробно на това.

2.3. Общи насоки за настройка на програмите

1. Не допускайте никоя програма да стане много голяма – разбивайте я на малки части, които могат да бъдат настройвани и тествани независимо.
2. Мислете за етапа на настройка още когато пишете програмата, за да не напишете програма, която е невъзможно да се настрои или тества.
3. Направете си списък на променливите и константите, използвани в програмата.

4. Разграничавайте проблемите и ги обработвайте един по един. Ако правите няколко промени едновременно и възникне грешка, нормално ще е да не разберете коя промяна е причинила новата грешка; също и няма как да разбере кое е поправило старата грешка.
5. Запомнете коригираната грешка и причината, която я е предизвикала, особено ако е необикновена. Така следващия път ще знаете какво е причинило тази грешка.

6. Не хвърляйте вината върху изчислителната машина.(Тя е много надеждна).
7. Имайте предвид, че винаги в програмата ще има грешки и че дори и добрите програмисти правят грешки. Но **добрите програмисти знаят**, че ще има грешки. Начинаещите смятат априори, че в тяхната програма по принцип няма грешки.
8. За да няма бъркотия, когато правите многобройни разпечатки в процеса на настройката, снабдявайте всяка една от тях с дата и час. Изхвърляйте ненужните листинги.

9. След корекция на една грешка прегледайте внимателно резултатите и програмата, за да се убедите, че други грешки не са били въведени.

10. Избягвайте съмнителното кодиране.

Предполагайте, че най-новите възможности не работят освен ако знаете, че работят. Използвайте най-прости оператори. Не се опитвайте да залъгвате транслятора или системата с хитри трикове. (Това после, след като тръгне програмата и ако е необходимо!).

11. Избягвайте зависимостта от “премълчаването”.

Всички езици за програмиране имат някакви възможности за премълчаване на неща, които транслаторът подразбира. Използването на тези възможности спестява работа на програмиста, но може да бъде опасно понеже производителите на компютри, ОС и среди за програмиране понякога променят тези възможности. Нещо повече, програмистът може да предполага, че има на разположение несъществуваща възможност.

12. Никога не допускайте зависимост от данните. Не позволявайте вашата програма да зависи от потребителя. Използвайте всички възможни механизми за проверка коректността на данните по време на въвеждането, за да сте сигурни, че те са правилни. Данните следват винаги закона на Мърфи: “Всичко, което може да откаже, отказва!”

13. Винаги завършвайте своите логически проверки докрай. Ако се допуска данните да имат стойност 1 или 2, не предполагайте, че стойността е 2, след като сте проверили, че стойността не е 1. Това пропуска патологичните случаи, които са чести. Винаги проверявайте за 1; ако не е 1, проверете за 2. Ако не е 1, нито 2, тогава програмирайте патологичния случай, т.е. обикновено съобщение за грешка или спиране на изпълнението.

3. Тестване на софтуер

- Тестването е процес на **изследване** или **изпълнение на програмата** с идеята да се намерят грешки.

3.1. Каква е целта на софтуерното тестване?

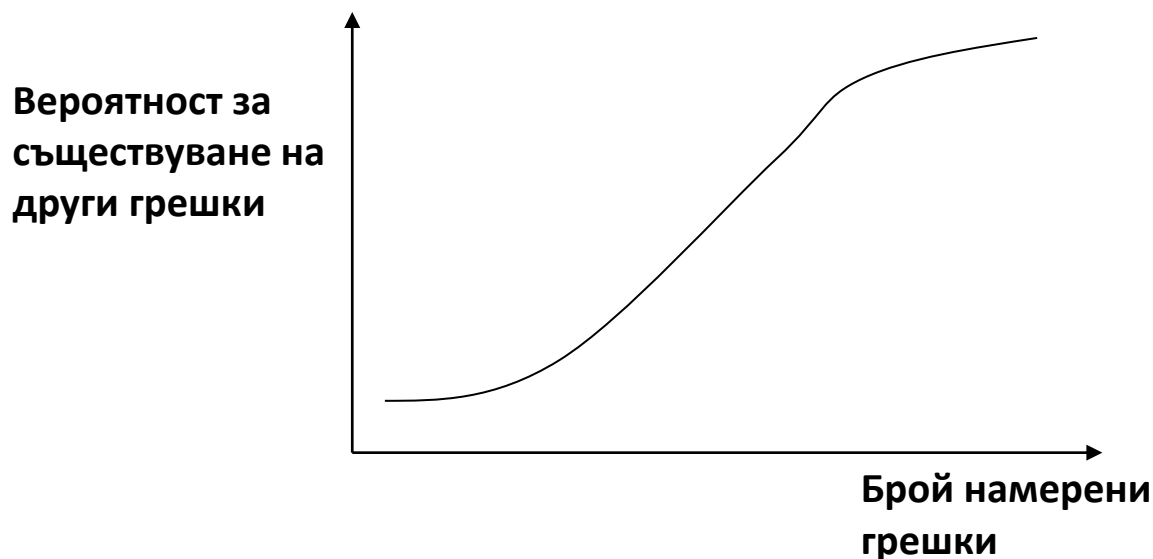
- а. Да установи наличието на дефекти в системата. То не може да докаже липсата на дефекти, може единствено да докаже, че са налице дефекти.**
- б. Да докаже, че софтуерният продукт изпълнява всяка предвидена функция;**
- в. Да докаже, че функционирането на софтуерния продукт се изпълнява съгласно спецификацията на изискванията и че всички нейни вътрешни компоненти са били адекватно създадени;**
- г. Да се увеличи доверието в правилното функциониране на софтуера.**
- д. Всичко, изредено по-горе.**

3.2. Аксиоми на тестването

(фундаменталните принципи за тестването)

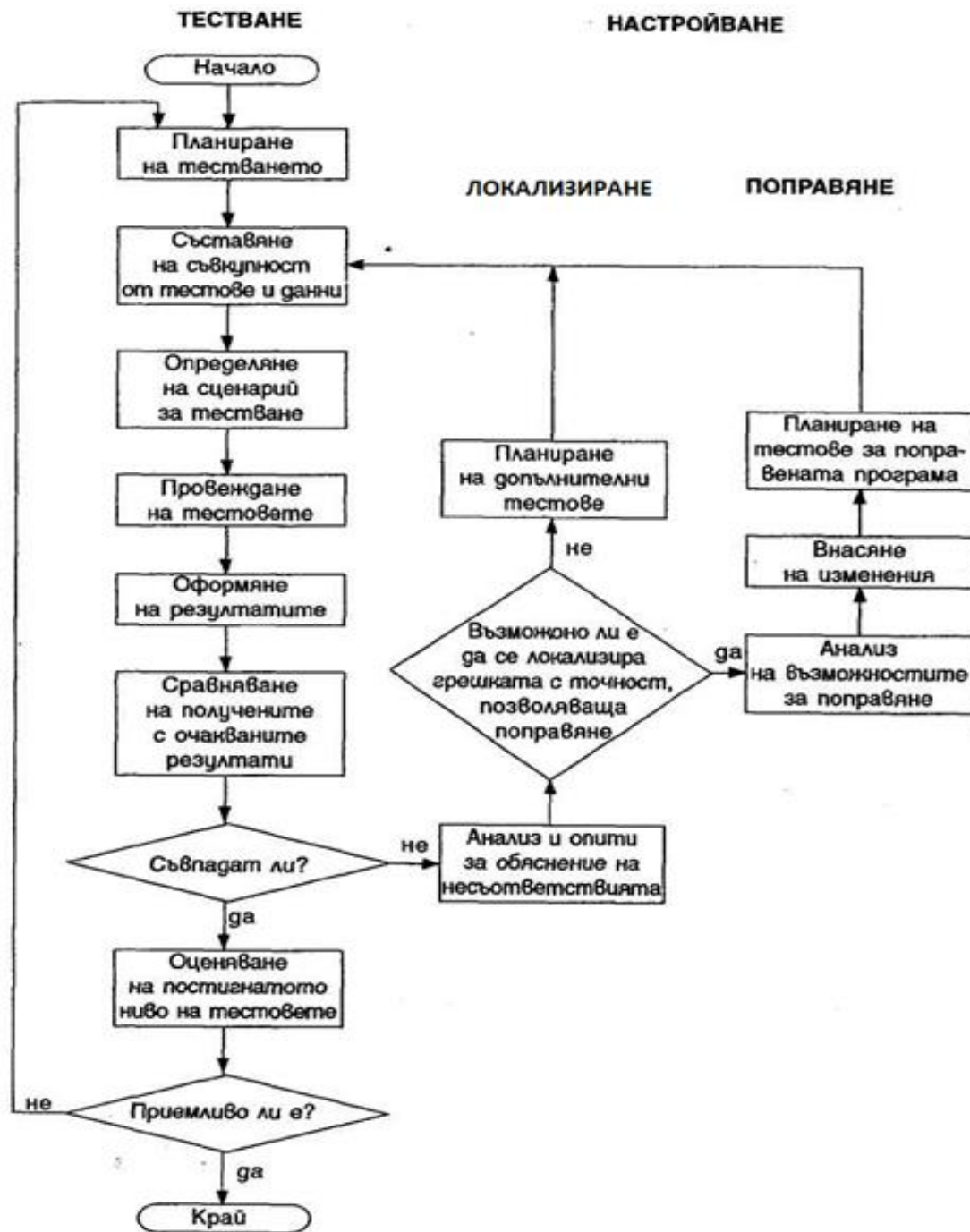
- 1. Добър е този тест, който притежава висока вероятност за откриване на грешки, а не този, който демонстрира правилната работа на програмата. “Успешен” е този тест, който намира поне една, все още неоткрита грешка.**
- 2. Не е възможно изчерпващо тестване.**
- 3. Невъзможно е да се тества собствената програма.**

- 4. Необходима част от всеки тест трябва да бъдат очакваните изходни данни или резултати.**
- 5. Трябва да се избягва тестването "ad hoc" ("от ход").**
- 8. С увеличаването на броя на грешките, откривани в дадена компонента на програмното осигуряване, се увеличава също и вероятността за съществуването в нея на неоткрити грешки.**



- 9. Тестването да се възлага на най-способните програмисти.**
- 10. Тестуемостта трябва да е ключова задача на разработката на софтуера.**
- 11. Никога не променяйте програмата, за да облекчите тестването.**
- 12. Тестването, както и почти всяка друга дейност, трябва да започва с постановка на целите.**

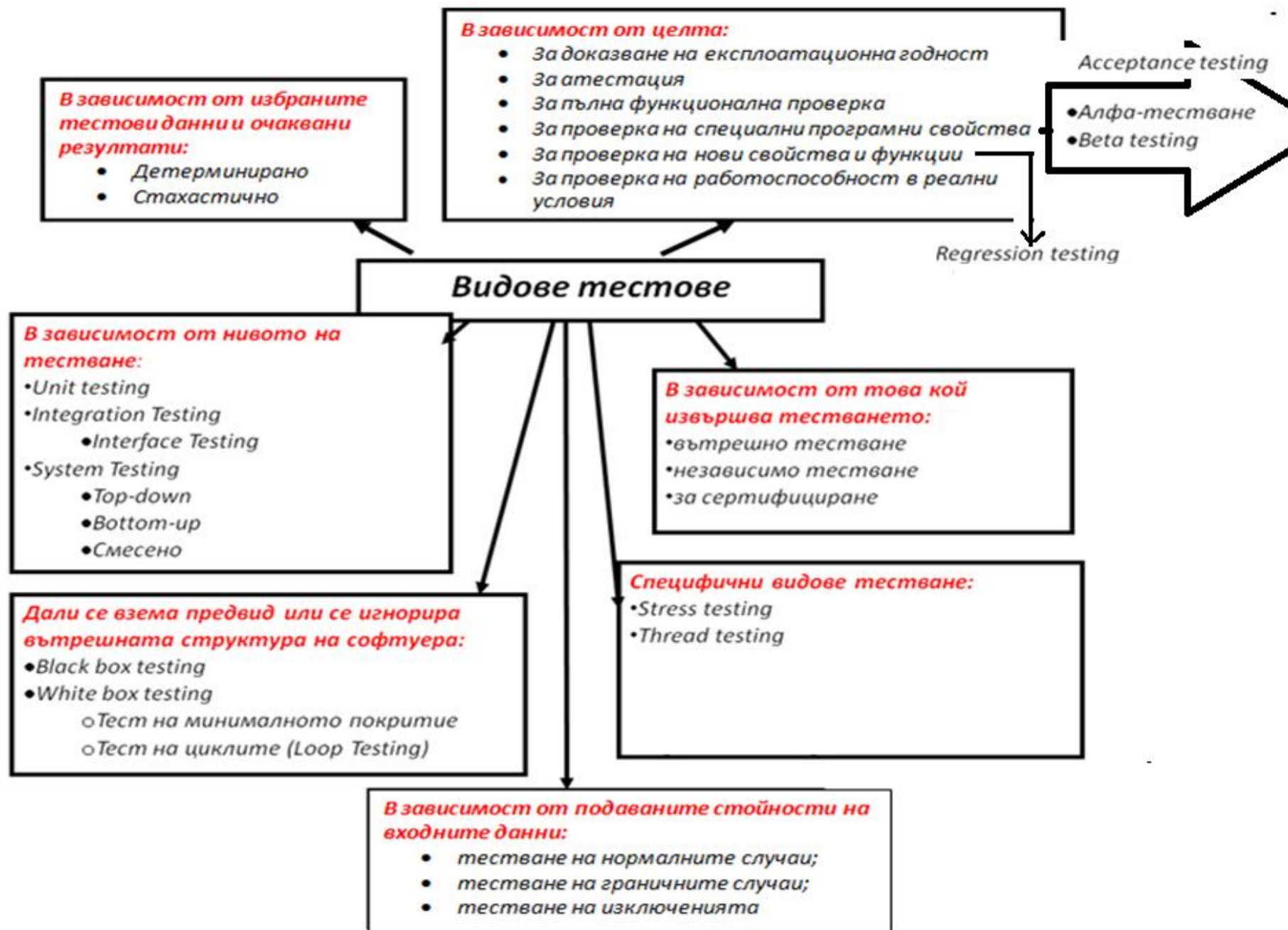
Да видим отново
връзката между
тестване и
настройка:



- Вижда се, че тестването включва 3 основни стъпки:
 1. **Планиране:** описва се какво да се тества, кога, с какви данни, кой да тества и как (**какви подходи да използва?**)
 2. **Реализация (Провеждането на тестването):** става на база на подготвени тестови данни и описани сценарии за тестване.
 3. **Отчитане:** описва се чрез сравнителен анализ на документираните резултати с очакваните и с оценяване на постигнатото ниво на тестване.

Един от най-сложните проблеми е да се оцени постигнатото ниво на тестване, т.е да се определи докога да се тества. Обикновено тестването завършва при **изтичане на предвидения срок** или при **изчерпване на ресурсите**.

Специалистите по тестване твърдят, че „**недостатъчното тестване е престъпление, а прекомерното — грях**".

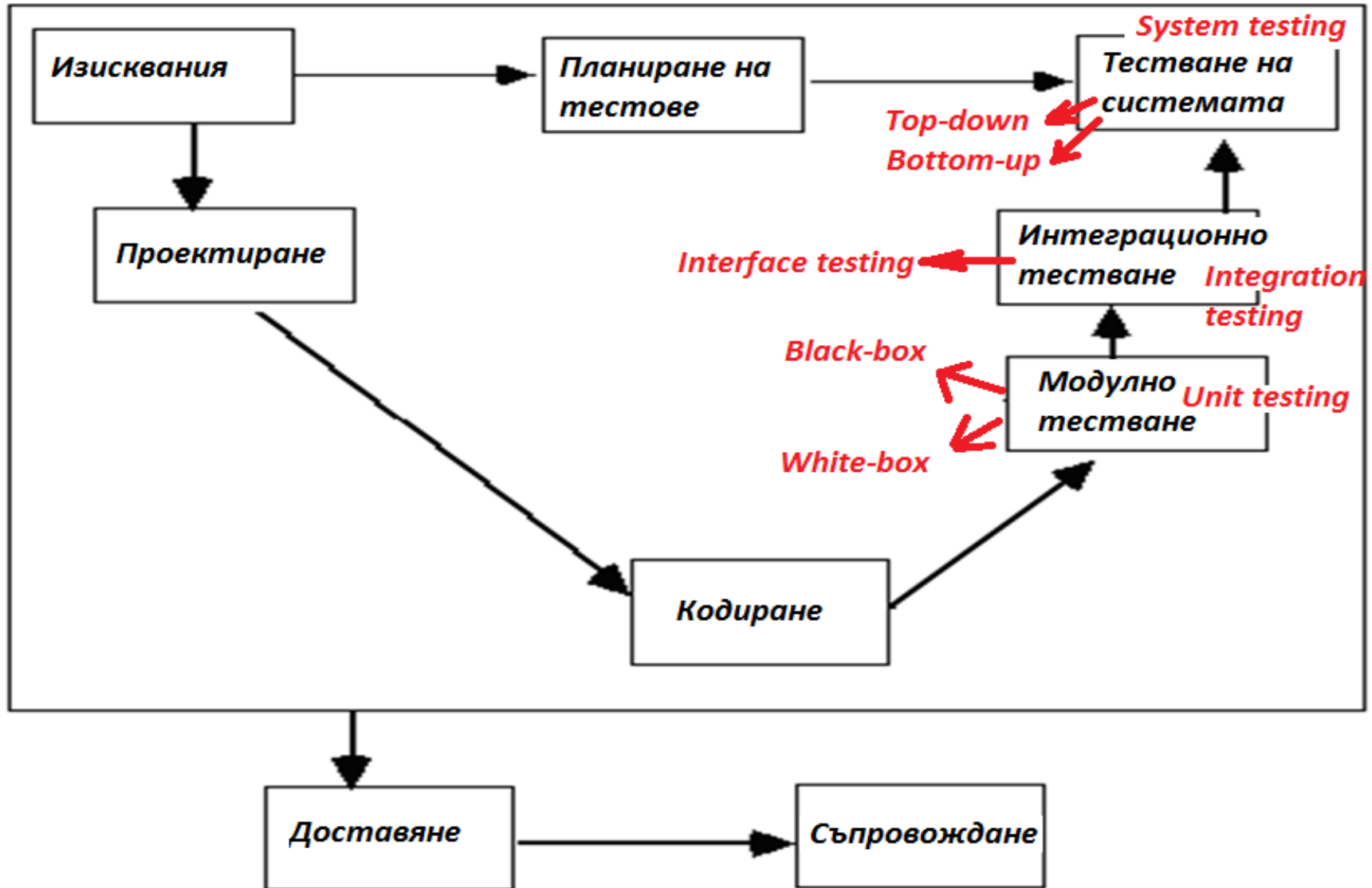


4. Методи за тестване.

Не съществува единна класификация на методите за тестване.

Темата разглежда различни (често пресичащи се) класификации на методите за тестване, базирани на различни критерии.

4.1...В зависимост от нивото на тестване



Unit testing

Тестване на ниво на компоненти (Unit testing).

- Извършва се от програмиста.
- Основни методи: Black- и white-box testing (функционално/структурно тестване).
- Спецификация: Тестващият се опира на изходния код на тестваната компонента или на група от свързани компоненти (т.е функция, клас, модул, компонент).

Забележка: Black- и white-box testing - Вж. т. 4.3.

Integration testing

Интеграционно тестване (Integration testing).

Тестване на съвместната работа на група от компоненти (модули) в една под-система.

- Извършва се от независим екип за тестване (testing team).
- Основни методи: Top-down и Bottom-up.
- Спецификация: тестерите гледат проектната спецификация на високо и ниско ниво (high- и low-level design documents).

Стратегии за тестване: Top-down и Bottom-up

- ✓ **Top-down testing (Низходящо).** Започва се с тестване на компонента на най-високо ниво, с постепенна интеграция на компонентите на по-ниско ниво, замествайки с драйвери виканите компоненти.
- ✓ **Bottom-up testing (Възходящо).** Тестване отдолу-нагоре. Интегриране на отделните компоненти в нива, докато се създаде цялостна система.

На практика, тестването е комбинация от тези стратегии.

Тестване на Интерфейса (Interface testing)

- Извършва се, когато модулите или под-системите се интегрират, създавайки по-голяма под-система, т.е вижда се, че **Interface testing** се извършва по време на **Integration testing** .
- Целта е да се открият дефекти, свързани с интерфейсни грешки или невалидни предположения за интерфейса.

Тестване на Интерфейса (Interface testing)

Видове интерфейси

1. **Parameter interfaces** (Параметрични интерфейси).
Данните се предават от една процедура към друга.
2. **Shared memory interfaces** (Интерфейси, базирани на обща памет). Блок памет е споделена между процедурите.
3. **Процедурни интерфейси**: Под-системата капсулира набор от процедури, които се извикват от други подсистеми.
4. **Message passing interfaces** (Интерфейси, свързани с предаване на съобщения). Подсистемите изискват услуги от други подсистеми.

Тестване на Интерфейса (Interface testing)

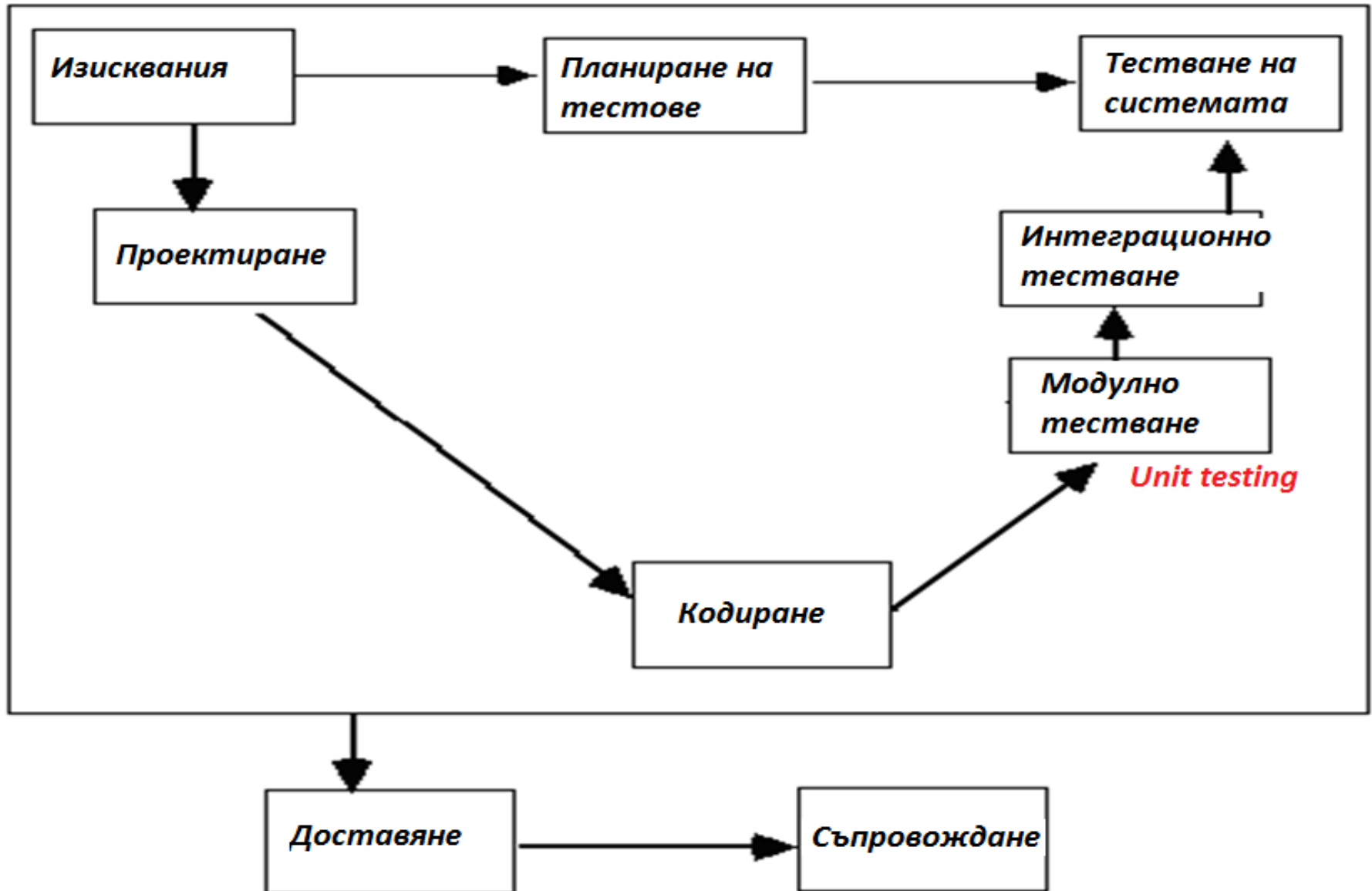
Интерфейсни грешки

1. Interface misuse (Интерфейсна злоупотреба). Един компонент извиква друг компонент и прави грешка при използването на интерфейса му напр. параметрите са в грешен ред.
2. Interface misunderstanding (Интерфейсни недоразумения). Извикващият компонент има неправилно предположение за поведението на викания компонент.
3. Timing errors (Временни грешки). Извиканият и извикващият компонент работят при различни скорости и с не-актуализирани данни.

Тестване на системата (System Testing)

- Системното тестване се явява **контрол или верификация**, ако се изпълнява в моделирана среда, и/или **изпитване или валидация** – ако се изпълнява в реална, жизнена среда.
- Основни методи: Top-down и Bottom-up.
- Спецификация: тестерите гледат проектна спецификация на високо ниво (high-level design document и software requirements specification).

Видове тестове: Място в ЖЦ



Специфични видове тестване на системата

Тестване под натиск (Stress testing)

- Това тестване и следващото се прилагат за определени класове системи. Те могат да се разглеждат като едно разширение и допълнение на системното тестване.
- Стресовото тестване е особено нужно за разпределените системи за обработка. Тези системи често проявяват известна ненадеждност, когато те са по-сериозно натоварени.
- Стресовото тестване може да се приложи със следните стратегии: отгоре надолу, отдолу – нагоре, или някои от останалите разновидности.

Специфични видове тестване на системата

Тестване чрез проследяване – нишково тестване (Thread testing)

- Друг специфичен клас системи, изискващ по-специални подходи при тестването, са системите, работещи в реално време. Те съдържат определено количество процеси, протичащи едновременно, които могат да бъдат прекъсвани при различни обстоятелства. Едно външно събитие, като например получаване на информация от датчик може да предизвика прекъсване на текущо изпълнявания процес с цел – обслужване на събитието.
- Системите в реално време са трудни за тестване поради неопределеността и зависещите от времето взаимодействия между процесите в системата. Зависещите от времето дефекти могат да предизвикат излизане на системата от строя, когато процесите и обработките се намират в някакво особено състояние.

4.2. Класификация на методите за тестване в зависимост от това, кой извършва тестването:

- **вътрешно тестване** — от самите разработчици. Всеки програмист проверява правилността на създадените от него програми.
 - Преимущество на този подход е, че програмистите познават програмите си, т.е. не е необходимо време за изучаването им.
 - Недостатъкът е, че програмистите имат склонност към надценяване на възможностите си, те са уверени, че не допускат грешки, а това пречи на системното тестване. Начин за преодоляване на този недостатък е т. нар. **peer review** — форма на вътрешно тестване, при която програмистите са разделени на двойки и всеки проверява програмите на партньора си.

- **независимо тестване** — от експерти, които не са участвали в разработването на програмите. Обикновено това са членове на групата по тестване, действащи в софтуерната фирма.
- **тестване за сертифициране** — проверка и издаване на сертификат, удостоверяващ наличието на определени свойства на ПП. Сертифицирането може да се извършва само от фирми, притежаващи съответния лиценз.

4.3. Класификация на методите за тестване в зависимост от това дали се взема предвид или се игнорира вътрешната структура на софтуера:

Има два основни метода за тестване: функционален и структурен.

Функционален метод (Black box testing – Метод на Черната кутия).

- При това тестване се проверява правилността на реализираните основни функции. То игнорира вътрешната структура на системата или на компонент от нея и се съсредоточава единствено върху изходните резултати, генерирани в отговор на избрани входни данни и условия за изпълнение.

Структурен метод (*White box testing – Метод на Бялата кутия/ Стъклена кутия*)

- При *структурното* тестване (на бялата кутия или „стъклена“ кутия) се съсредоточава върху вътрешната структура на системата или на компонент от нея.
- Целта на *структурното тестване* е да се изберат такива тестови данни, че да се осигури преминаване през всички програмни части на тествания компонент или на системата.
- Използва се най-вече за проверка на **управляващата структура** на процедурите, тоест за unit testing.

Структурен метод (*White box testing, т.е тестване на Бялата /Стъклена кутия*)

Последователността от тестове трябва:

- 1) да гарантира, че всички независими пътища, във всеки модул, са тествани (изпълнени) най-малко веднъж – **“Тест на минималното покритие”**;
- 2) да изпълни всички логически условия и в двете посоки: и при true и при false;
- 3) да изпълни всички цикли, включително и за техните гранични стойности (**“Тест на циклите (Loop Testing)”**);
- 4) да провери вътрешните структури от данни, за да се гарантира тяхната валидност.

Тест на минималното покритие

- Целта на този подход е да се тестват всички “независими пътища” в програмата, поне веднъж.

Реализация на метода:

- Отправната точка за тестването е **управляващия граф** (граф на потока) на програмата: възлите представят управляващите оператори на програмата, а дъгите - потока на управление между управляващите оператори.
- За да тествате всички независими пътища поне веднъж, броят на тестовете ще е равен на цикломатичната сложност на графа:

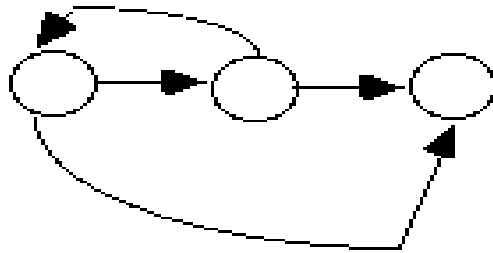
Цикломатична сложност = Брой на дъги – Брой на възли + 2

- !!! Този тест гарантира тестване на всички независими пътища поне веднъж, но не гарантира тестване на всички комбинации от независими пътища.

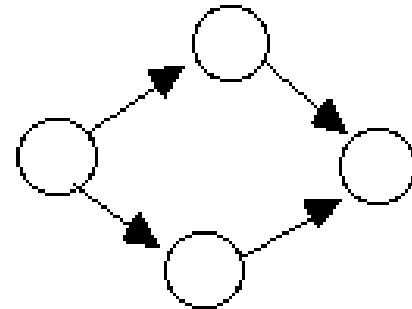
Тест на минималното покритие - стъпки

1. Представяне на управляващите оператори на програмата и създаване на управляващия граф:

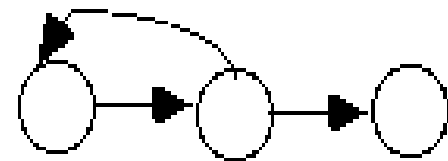
While



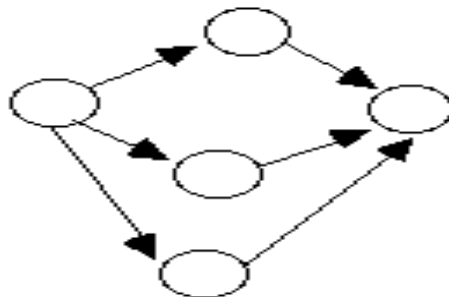
IF



Repeat



Case



2. Определя се мин.брой тестове, гарантиращи изпълнение на всички независими пътища, поне веднъж.
- Броят на тестовете, ще е равен на цикломатичната сложност $V(G)$ на програмата:

Цикломатична сложност =

$$\text{Брой на дъги} - \text{Брой на възли} + 2$$

3. Определят се независимите пътища и последователността от тестове (тестова последователност), която да ги покрие.

Стъпка 1: Създаване на Управляващ граф

- a) идентифицирайте условните възли (predicate nodes)
- b) създайте управляващия граф (flow graph)

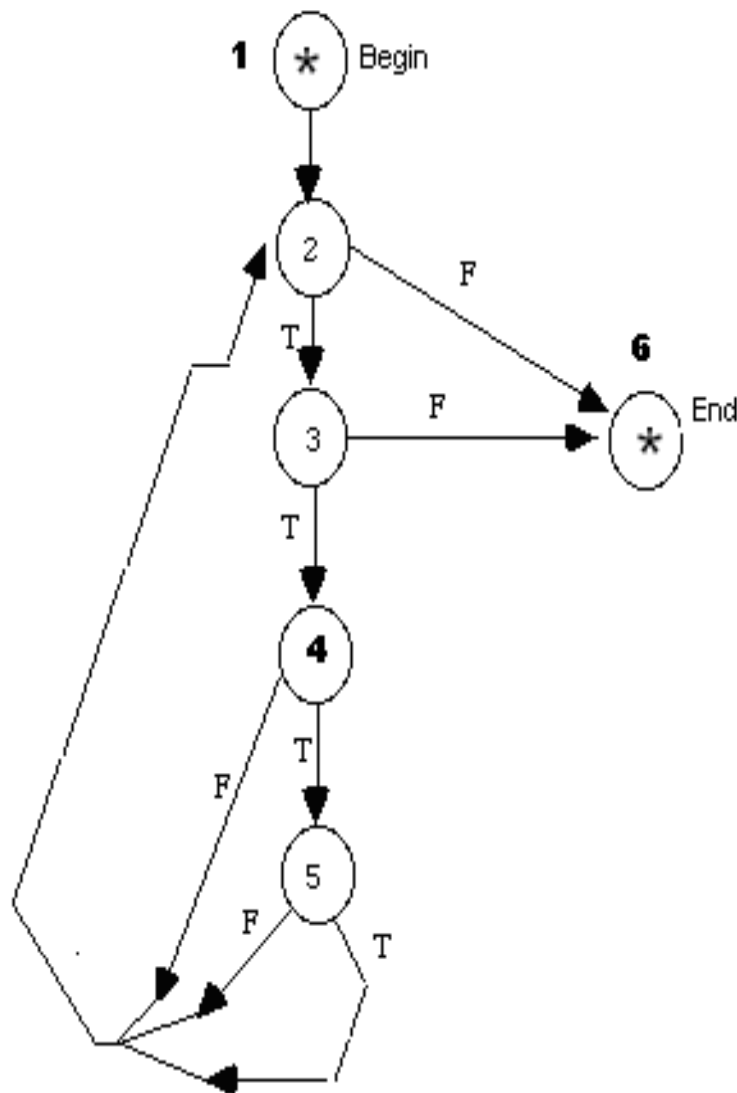
Стъпка 2: Изчислете цикломатичната сложност

$$V(G) = E - N + 2,$$

$$\text{T.e. } V(G) = 9 - 6 + 2 = 5,$$

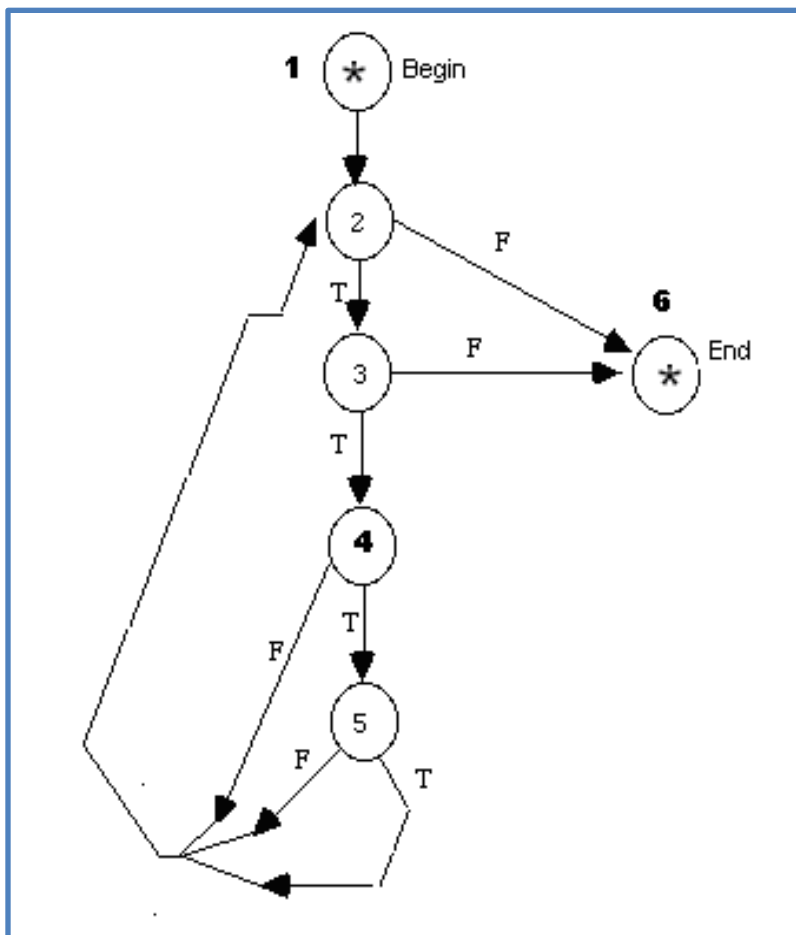
Следователно, минималният брой тестове, гарантиращи тестване на всички независими пътища поне веднъж, е равен на 5.

Пример:



Пример:

Стъпка 3: Създайте тестова
последователност:



Път

1-2-6

1-2-3-6

1-2-3-4-2-6

1-2-3-4-5-2-6

1-2-3-4-5-2-6.

Тест

1

2

3

4

5

Тест на циклите (Loop Testing)

Обикновени цикли Следната последователност от тестове може да бъде приложена спрямо цикъла, където n е максималния брой от позволени тестове:

1. Прескача се цикъла изцяло.
2. Само веднъж се минава през цикъла.
3. Два пъти се минава през цикъла.
4. m пъти се минава през цикъла, където $m < n$.
5. $n-1$, n минавания през цикъла

Вложени цикли

1. Започнете с тестване на най-вътрешния цикъл. Тествайте всички други цикли минимално (с минимален брой итерация).
2. Тествайте външния цикъл на най-вътрешния, а всички други външни цикли тествайте минимално.
3. Продължете нататък, докато всички външни цикли са тествани.

- Забележка: Комбинацията: тест на минималното покрие, следван от тест на циклите, е позната като „тест на пълното покритие“.

4.4. Класификация на методите за тестване, в зависимост от избраните входни данни:

- *стохастично тестване (Random тестване)* - техника за тестване на софтуер по метода на черната кутия, при която програмите се тестват чрез случайно генерирани входни величини.
- *детерминирано тестване* — програмата се тества чрез детерминирани (не случайни) входни величини.
- Спецификация: Software Requirement Specification. Тоест, получените изходни резултати се валидират, като се използва спецификацията на изискванията към софтуера. В случай на отсъствие на такава, се използва вградения в езика за програмиране механизъм на изключенията, което означава, че ако възникне изключение по време на изпълнение на теста, то това ще означава, че е налице грешка в програмата.

4.5. Класификация на методите за тестване, в зависимост от подаваните стойности на входните данни:

Фактическото тестване на една програма (независимо от използвания подход) може да бъде разделено на три фази:

- тестване на нормалните случаи;
- тестване на граничните случаи;
- тестване на изключенията

Ще разгледаме няколко метода за тестване, базирани на тази класификация:

- 1. Метод на еквивалентното разделяне (Equivalence partitioning) на входните данни (тестване на нормални случаи/изключения).**
- Цели да покаже, че всички валидни (правилни) данни дават правилни резултати и всички невалидни данни дават винаги съобщение за грешка.
 - За целта:
 - входните данни се представят като множество от еквивалентни класове (определят се еквивалентни класове за всяка променлива).

За метода на еквивалентното разделяне (Equivalence partitioning) на входните данни

- Един еквивалентен клас представлява множество от **валидни** или от **невалидни** данни, като по този начин се позволява тестване съответна на **нормалните случаи и на граничните стойности** една страна и на **изключенията** – от друга.
- Класовете се определят по следния начин: стойностите които са допустими за една променлива, съответстват на **правилен еквивалентен клас**, а тези които са недопустими - **неправилен еквивалентен клас**.

За метода на еквивалентното разделяне (Equivalence partitioning) на входните данни.

Насоки за прилагане на метода:

- 1) Ако стойностите на една променлива (например „a“) трябва да са в някакъв диапазон, например $[1...10]$, то за „a“ има един валиден, т.е. един правилен клас ($a \in [1...10]$) и 2 невалидни или неправилни класа ($a < 1$ и $a > 10$).
- 2) Ако една променлива трябва да има точно определена стойност, то тя има един правилен и 2 неправилни класа.

За метода на еквивалентното разделяне (Equivalence partitioning) на входните данни.

- 3) Ако една променлива специфицира член на множество, то тя има един правилен и 1 неправилен клас .
 - 4). Ако една променлива е от тип **boolean**, то тя има един правилен и 1 неправилен клас.
- Вариантите за тестване би трябвало да покриват всички класове.
 - В контекста на 3-те фази на тестване, коментирани по-горе, този тест подкрепя фазите тестване на нормалните случаи и на изключенията.

За метода на еквивалентното разделяне (Equivalence partitioning) на входните данни.

Пример:

- Ако променлива “a” е трицифрено число между 200 и 999 (включително, т.е $a \in [200...999]$), еквивалентните класове за “a” са:
 - 1) валиден: $a \in [200...999]$, т.е „a” е между 200 и 999 (включително)
 - 2) невалиден: $a > 999$
 - 3) невалиден: $a < 200$
- Избирайте данни от всяки един клас, близки до граничните:
a=300; a=1000; a=199.
- Целта е, с минимален на брой тестове да се покрият всички еквивалентни класове.
- Неправилните класове се тестват обаче поотделно.

2. Метод на граничните значения (Boundary Value Analysis)

- Този метод е допълнение към предния, тестващият избира стойности на границите на входната област, които трябва да се възприемат като валидни данни.
- Ако променлива “a” е трицифрено число между 200 и 999 (включително), то се тества със стойности:

a=200 и a=999.

- В контекста на 3-те фази на тестване, коментирани по-горе, този тест подкрепя фазата «тестване на граничните случаи».

4.6. Класификация на методите за тестване в зависимост от начина за интеграция на модулите в системата

В зависимост от начина за интеграция на модулите в системата, тестването може да е:

- **низходящо (top down) тестване,**
- **модифициран низходящ метод на тестване,**
- **метод на големия скок,**
- **възходящо (bottom up) тестване,**
- **смесено тестване (метод на сандвича) ,**
- **модифициран метод на сандвича.**

Низходящо (top down) тестване

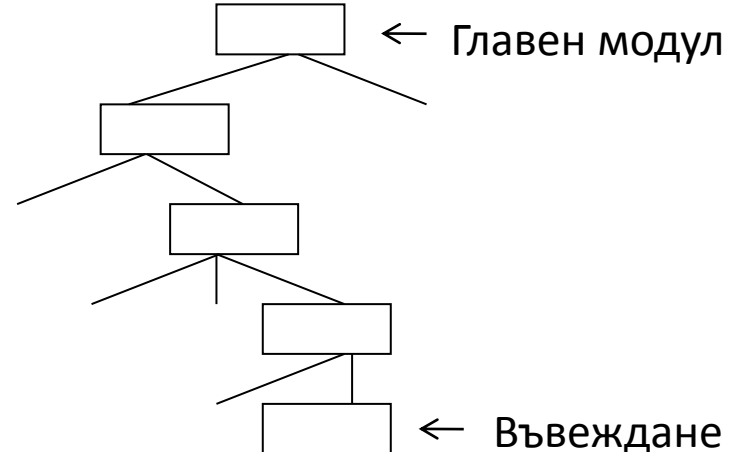
Ще припомним:

- При низходящия подход програмата се тества (и разработва) отгоре надолу:
- **Изолирано се тества само главният модул.**
- След като тестването на главния модул е завършено, към него се съединяват един след друг модулите, непосредствено извиквани от главния модул и се тества получената комбинация.
- Процесът се повтаря, докато се сглобят и проверят всички модули.

Низходящо (top down) тестване

При този подход веднага възниква въпроса.

- Какво да се прави, когато тестваният модул извиква модул от по-ниско ниво, който в дадения момент все още не е разработен?
- Този въпрос се решава чрез имитиране на функциите на недостигащите модули с помощта на **модули-драйвери**.



Низходящо (top down) тестване

Предимства на низходящото тестване.

- A. Методът съвместява тестването на модул (unit testing), с тестване на свързването (integration testing).**
- B. Той е удобен за случаите, когато имаме съмнения за осъществимостта на програмата, така получаваме отговор на този въпрос достатъчно рано.**

Недостатъци.

- A. Модулите рядко се тестват детайлно (фундаментално) преди включването им към системата, тоест не може да се каже, че извършеното „unit testing” е качествено.**
- B. Вторият тънък недостатък на низходящото тестване се състои в това, че той може да породви вяра във възможността да се започне програмиране и тестване на горното равнище от програмата, преди да е проектирана цялата програма. Тази идея на пръв поглед ни се струва добра (икономична), но обикновено работата стои съвсем обратното. Повечето от опитните програмисти знаят, че проектирането на програмата е итеративен процес. Рядко, първият вариант, се оказва най-добър.**

Модифициран низходящ метод на тестване

- Една от слабите страни на низходящото тестване се състои в недостатъчната пълнота на тестването на подчинените модули.
- Методът, който се нарича модифициран низходящ подход, решава този проблем.
- Изисква се всеки модул да е преминал автономно тестване („unit testing”), преди да се подключи към програмата.

Метод на големия скок

- Същината му се свежда до автономното тестване на всеки модул.
- След приключване на тестването на модулите, те се интегрират в системата всички едновременно.
- Този метод предполага, че към началото на тестването всички модули са готови.
- Макар, че на практика това е може би най-разпространения метод при тестването и интеграцията на модулите, той (сравнен с другите) има много недостатъци и малко предимства ...

Метод на големия скок

Основните недостатъци се свеждат до:

- не способства за съкращаване времето за разработване на приложението, защото тестването започва едва след завършването на всички модули;
- модулите не се интегрират до последния момент, което означава, че дълго време сериозни грешки от спрягането могат да останат неоткрити;
- при едновременно стиковане възникват много и най-разнообразни грешки.

Метод на големия скок

Предимства:

- **Не се налага да се пишат допълнително драйверни програми.** Той е приложим при изцяло разработени и готови модули и сравнително малки програми

Възходящо (bottom up) тестване

- При възходящото тестване, програмата се разработва и тества отдолу нагоре.
- **Само модулите на най-ниско ниво (“терминални” модули, неизвикващи други модули) се тестват изолирано, автономно.**
- След това се тестват модулите, които непосредствено извикват вече проверените, но не се тестват автономно, а заедно с вече проверените модули от по-ниското равнище.
- Процесът се повтаря дотогава, докогато не бъде достигнат върхът на йерархията.

***Възходящо (bottom up)* тестване**

- В процеса на сглобяването на системата може да се окаже, че някои от модулите от по-горните нива все още не са готови.
- За да не се задържа тестването, което се припокрива във времето с разработването, на мястото на незавършените модули се поставят така наречените **драйвери**. Това е неголяма водеща програма, чиято задача е единствено да генерира подходящи тестващи данни за подчинените модули.

Смесено тестване (метод на сандвича)

- Тестването по метода на сандвича представлява компромис между възходящия и низходящия подход. Той е опит да се използват достоинства на двата подхода, избягвайки недостатъците им.
- При използването на този метод едновременно започват възходящо и низходящо тестване, сглобявайки програмата както отгоре, така и отдолу, срещайки се някъде по средата. Точката на срещане зависи от самата тествана програма и трябва да бъде определена при изучаването на нейната структура.
- Този метод е разумен по отношение на интеграцията на големи програми, такива като операционни системи, приложни програмни продукти.

Модифициран метод на сандвича

- При тестването по метода на сандвича възниква същия проблем, както и при низходящото тестване, макар тук той да не стои така остро, а проблемът на низходящото тестване бе в това, че методът не предполага детайлно тестване на отделните модули.
- В модифицирания метод на сандвича този недостатък се отстранява – изисква се **долните нива също да се тестват строго, преди да бъдат прибавени към горните.**

4.7. Класификация на методите за тестване в зависимост от целта на тестването

- 1. за доказване на експлоатационната годност на ПО** - проверява се дали програмният продукт е работоспособен (Black box testing);
- 2. за атестация** — т. нар. пускови изпитания, след успешното завършване на които програмният продукт може да се разпространява;
- 3. за пълна функционална проверка** (acceptance testing) — тества се дали реализираните функции съответстват на формулираните изисквания;

Тестване за пълна функционална проверка (Acceptance testing)

Тук различаваме:

- ☐ Алфа-тестване и
- ☐ Бета-тестване

Алфа-тестване. След тестването на системата продуктът се предоставя на потребителите и те сами проверяват (в условията на средата за разработване) дали реализираните функции съответстват на формулираните изисквания. Основен метод: Black-box testing.

- Спецификация: Software requirements specification.
- Алфа-тестването се провежда с участието и на потребители, но в средата на разработване за разлика от Бета-тестването.

Beta testing (още installation testing) .

- Основен метод: Black-box testing.
- Спецификация: Software requirements specification.
- Същност: Когато е налице частична или пълна версия на софтуера, разработващата организация я предлага безплатно на един или много потенциални потребители или бета тестери. Прави се *проверка на работоспособността на системата в реални потребителски условия* (installation testing) — изследва се времето за отговор на системата, продължителността на входно-изходните операции, изчислителните ресурси; качеството на потребителския интерфейс и др.

Beta testing: Предимства:

- Идентификация на неочаквани грешки тъй като бета потребителите извършват неочаквани действия.
- Много на брой тестове за грешки в голямо разнообразие от среди (ОС и др.).
- Безплатно тестване.

Beta testing: Недостатъци:

- Липса на систематичност в тестването.
- Ниско качество на докладите за намерени грешки и големи усилия да се разчетат тези доклади.

4. **за проверка на *специални програмни свойства*,
например надежност или преносимост;**
5. **за проверка на *работоспособността на
системата в реални потребителски условия
(installation testing)* —** изследва се времето за
отговор на системата, продължителността на
входно-изходните операции, изчислителните
ресурси; качеството на потребителския интерфейс и
други.
6. **за проверка на *нови свойства или функции* —**
реализира се след внасяне на изменения в
създадения програмен продукт (**регресионно
тестване**);

Regression testing

- Това тестване е “**за проверка на нови свойства или функции**”. Изпълнява се след внасяне на изменения в програмния продукт.
- Основни методи: Black- и white-box testing.
- Спецификация: Всеки документ за промяна на софтуера; Проектната спецификация на най-високо ниво.
- Същност: това е селективно ре-тестване само на тези компоненти от системата, в които са внесени изменения. Идеята я да се да се уверим, че внесените промени не са довели до неочаквани ефекти и че компонентите работят все така, в съответствие с изискванията.

4.8. Кой метод за тестване на системата е най-добър?

- Трудно да се оценят описаните по-горе методи (подходи, стратегии) за тестване на системата и да се намери най-добрия, защото “най-добрият” подход зависи от конкретната организация и конкретната програма.
- От гледна точка на надеждността на ПО, различните методи за тестване могат да се оценят по следните критерии:
 - 1. Времето до момента на сглобяване**, тъй като това е важно за откриването на грешки в стиковката на модулите.

2. Времето до момента на създаването на първите, работещи “скелетни” версии на програмата, т.к. тук могат да се проявят главните дефекти на проектирането.
 3. Необходими ли са драйвери или други инструментални средства.
 4. Възможност за осъществяване на паралелно тестване.
 5. Възможност за тестване на отделните пътища.
 6. Възможност за планиране, управление и контрол на процеса на тестването.
- Ако изключим Thread testing и Stress testing, които имат специфични приложения, то на основата на дадените по-горе критерии, предпочитание следва да се даде на метода за **тестване отдолу-нагоре** и на **модифицирания метод на сандвича**.

5. Автоматизиране на тестването

Казахме, че инструменталните средства, подпомагащи настройването, се наричат **програми -дебъгери**.

В зависимост от предназначението си програмните средства за тестване могат да бъдат:

- **анализатори на програми** - реализират избран метод на тестване, чрез опериране върху тестващата програма по съответстващ на метода начин
- **генератори на тестови данни**,
- **среди за тестване**

Предимства на автоматизацията

- Чрез автоматизация на настройването и тестването се постига:
 - систематичност;
 - подобряване организацията на тестване;
 - повишаване надежността на програмната система;
 - документиране на извършваните дейности;
 - автоматично измерване обхвата на тестването.

7. Осъществяване на тестването

- Натрупаният практически опит показва, че резултатите от тестването установяват наличие на грешки и дефекти твърде късно — тогава, когато програмният продукт е почти завършен.
- Доказано е, че усилията E за отстраняване на грешките растат експоненциално, в зависимост от дължината на времевия интервал t , определен от момента на допускане на грешката t_1 , до момента на откриването на грешката t_2 :
$$t = t_2 - t_1 \qquad E = k \cdot e^t$$
- Затова, усилията са насочени към разработка на методи и техники, позволяващи ранното откриване и отстраняване на грешки.

”Намирането и отстраняването на грешки си е отговорност на програмиста.

Програмистите са тези, които допускат грешките и ги оставят да минат покрай тях. Така че тяхна е отговорността да предотвратяват изтичането на грешки към тестващата група и оттам към клиентите”.

Стив Магуайър (Microsoft)

Въпроси:

- *Какви дейности са настройване и тестване в процеса на създаване на софтуер?*
- *Какво е дефект в програмата?*
- *Какво е грешка в програмата?*
- *Какви видове грешки в програмата може да откриете?*
- *Кои са източниците на грешки в програмите?*
- *Каква е целта на софтуерното настройване?*
- *Каква е разликата между Тестване и Настройване?*

Въпроси:

- *Каква е разликата между Верификация и Валидация?*
- *Кои са най-общо подходите за локализиране на грешките?*
- *Какви насоки може да посочите за настройка на програмите?*
- *Каква е целта на софтуерното тестване?*
- *Кой тест се счита за добър?*
- *Кой тест се счита за успешен?*
- *Избройте и коментирайте аксиомите (основните принципи) на тестването?*
- *Какви са методите за тестване в зависимост от нивото на тестване?*

- *Какво е unit testing?*
- *Какво е Integration testing?*
- *Какви видове интерфейсни грешки има?*
- *Каква е разликата между Алфа-тестване - Бета-тестване?*
- *Какви са методите за тестване в зависимост от това, кой извършва тестването?*
- *Какви методи за тестване има в зависимост от това, взема ли се предвид или не вътрешната структура на софтуера?*

- *Какво представлява теста на минималното покритие? Какво наричаме тест на максималното покритие?*
- *Какви методи за тестване има в зависимост от избраните тестови данни?*
- *Какви са методите за тестване в зависимост от подаваните стойности на входните данни?*
- *Каква е целта на метода еквивалентно разделяне на входните данни? А на метода на граничните значения на входните данни?*
- *Какви са методите за тестване в зависимост от целта на тестването?*

- *Какви са методите за тестване в зависимост от начина за интеграция на модулите в системата?*
- *Коментирайте същността, предимствата и недостатъците на низходящото (top down) тестване и неговите модификации: модифициран низходящ метод на тестване, метод на сандвича и модифициран метод на сандвича.*
- *Коментирайте същността, предимствата и недостатъците на възходящо (bottom up) тестване.*
- *Кой метод за тестване на системата е най-добър?*

Използвана Литература

1. Нели Манева, Аврам Ескенази, Софтуерни технологии.
Анубис София 2001
2. Нели Манева, Аврам Ескенази, Софтуерни технологии.
КЛМН, София 2006
3. Sommerville: Software Engineering ,
9. ed. Addison-Wesley, 2011,
[http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20\(intro%20txt\)%20-%20I.%20Sommerville%20\(Pearson,%202011\)%20BBS.pdf](http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20(intro%20txt)%20-%20I.%20Sommerville%20(Pearson,%202011)%20BBS.pdf)
4. https://bg.wikipedia.org/wiki/%D0%A2%D0%B5%D1%81%D1%82%D0%B2%D0%B0%D0%BD%D0%B5_%D0%BD%D0%B0_%D1%81%D0%BE%D1%84%D1%82%D1%83%D0%B5%D1%80